AI Customer Support Platform

TECHNOLOGY OVERVIEW

The technologies behind the platform

What we use, the role each one plays, the rule for using it, and what it could be swapped for.

→ next chapter · ↓ details inside a chapter · Esc overview · I all decks · click a diagram to enlarge

CHAPTER 1

How we choose

Criteria first, products second. Every name here is an example of a category.

01

STATUS

A proposal, confirmed by decisions

Each technology is a **recommendation**. It becomes a decision only through an **ADR**, with evidence from a spike or an evaluation.

- Provider-specific parts live in adapters, never in the core domain
- Vendor names, prices, licenses and data policies are verified at ADR time
- If the team prefers Python, only the language rows change, not the architecture

SELECTION CRITERIA

Six tests every choice must pass

Fits the team
Few moving parts, one main language and database

Proven where it matters
Mature storage, queues, auth and delivery

Security first
Cross-tenant access is hard by design

Portable core
No vendor type inside the domain

Observable and testable
Every decision and tool call can be traced and replayed

Reasonable cost
Start simple, keep a trigger for moving up

ASSUMPTIONS

What we assume today

- A small team, text MVP first, voice and operations later
- TypeScript as the primary language
- Cloud-agnostic containers, managed services welcome, no design tied to one cloud
- One channel (website chat) and exactly one read-only business tool in the first release

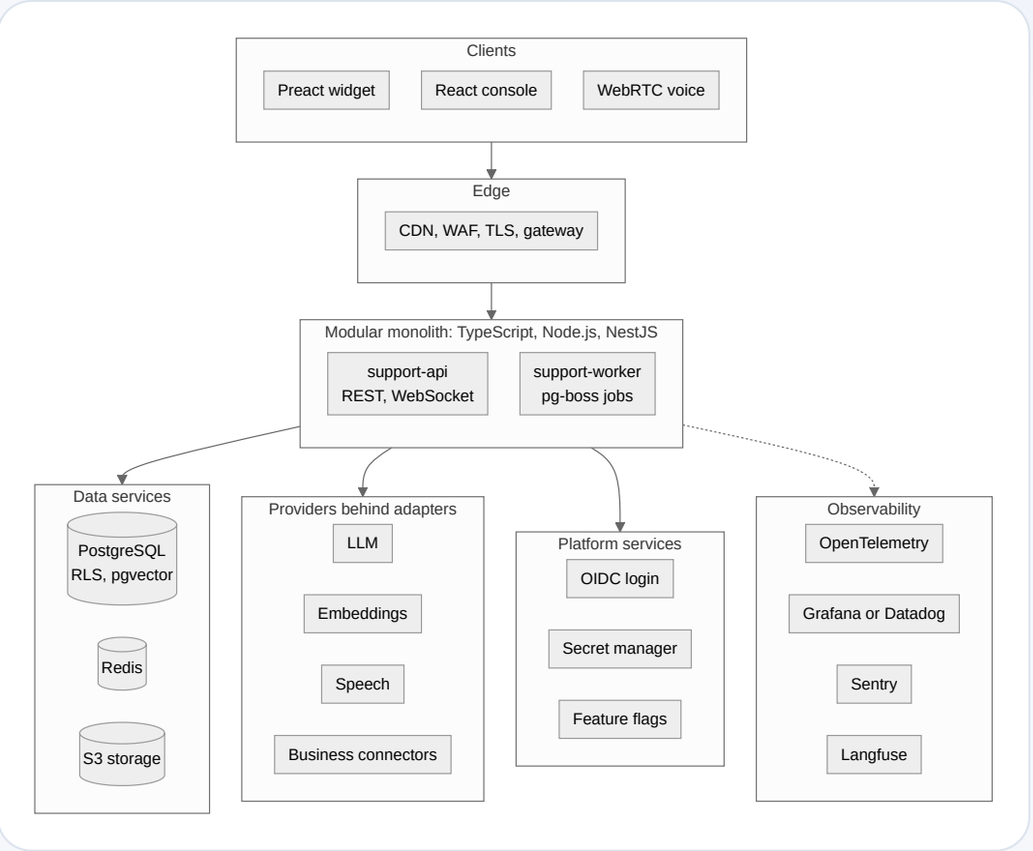
CHAPTER 2

The stack at a glance

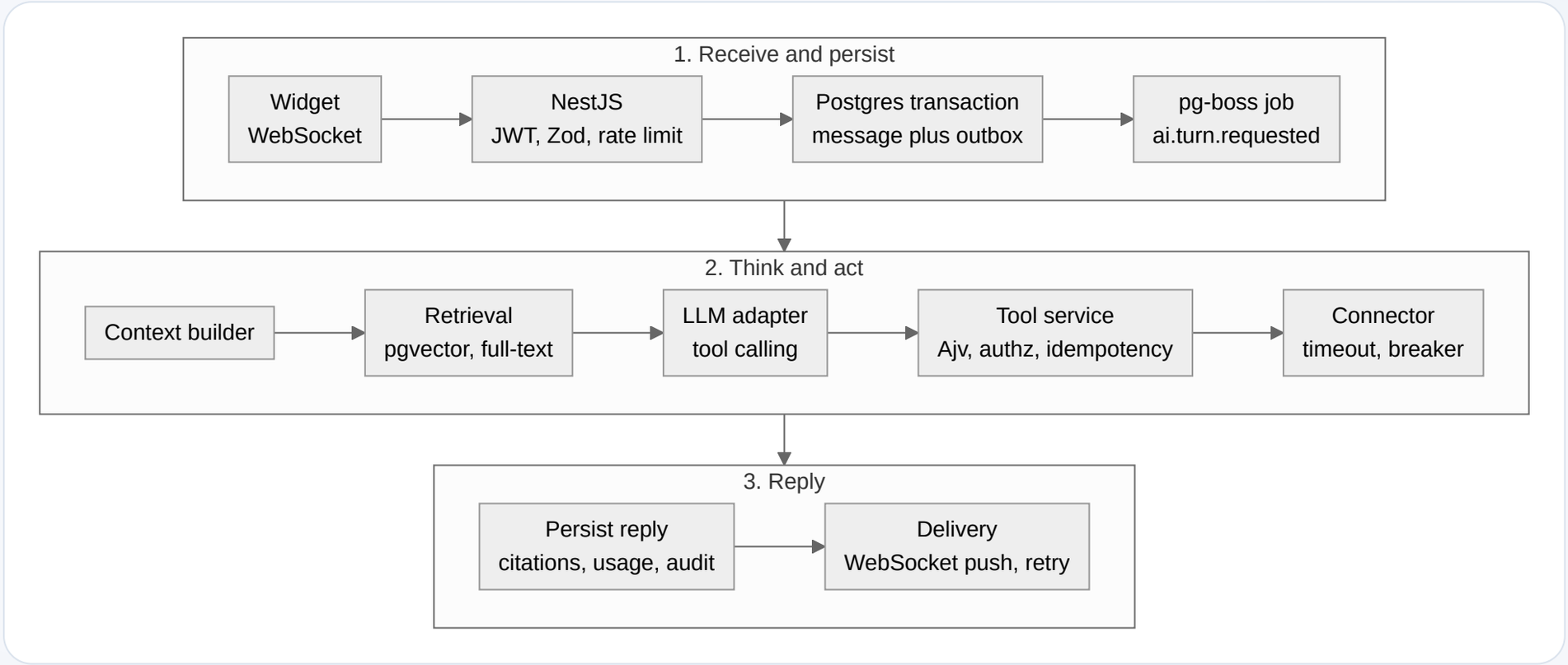
Seven layers, one modular monolith in the middle.

The stack in one picture

- **Clients:** widget, console, browser voice
- **App:** one TypeScript codebase, API and worker
- **Data:** PostgreSQL does most of the work
- **Providers:** models, speech and business systems behind adapters
- **Platform:** login, secrets, flags
- **Observability:** traces, metrics, errors



One message through the stack



No database transaction is held open while the model or a business system is called

CHAPTER 3

Backend and data

One language, one database, and rules that keep tenants apart.

APPLICATION AND STORAGE

Core building blocks

TypeScript and NestJS

Role application code and module boundaries

Rule modules talk only through interfaces, never through internals

Instead of: Python with FastAPI

PostgreSQL 16

Role system of record, queue, outbox and vector index

Rule no transaction stays open during an external call

Instead of: MySQL, CockroachDB

Row-level security

Role tenant isolation enforced by the database

Rule tenant context is set per request, and a missing context returns nothing

Instead of: schema per tenant

SEARCH, JOBS AND FILES

Retrieval, work queues and objects

pgvector and full-text

Role knowledge retrieval

Rule every query carries tenant and source filters

Instead of: Qdrant, OpenSearch

pg-boss

Role durable jobs, retries and delayed jobs

Rule enqueue in the same transaction as the change, and keep consumers idempotent

Instead of: BullMQ, RabbitMQ, SQS

Redis and S3 storage

Role rate limits and short-lived data, documents and media

Rule nothing durable lives only in Redis, objects are private with signed URLs

Instead of: in-memory, GCS or Azure Blob

CHAPTER 4

The AI stack

Models, retrieval, contracts and the evidence that they work.

04

MODELS AND RETRIEVAL

How the AI gets its answers

LLM adapter

Role model access with routing

Rule vendor types never leave the adapter, and model, latency and usage are recorded

Instead of: OpenAI, Google, self-hosted

Embedding adapter

Role text vectors for retrieval

Rule index and query use the same model version

Instead of: Voyage AI, OpenAI, BGE or E5

Hybrid retrieval

Role find the evidence

Rule vector plus keyword, fused and thresholded, with a reranker only if evaluation shows a gain

Instead of: a dedicated search engine

MODEL ROUTING, DECIDED BY ADR-005

Right-sized model for each job

Small model

Intent and risk classification, rolling summaries, handoff bundles.

Mid-size model

Customer replies, grounded in retrieved evidence and verified tool results.

Large model

Hard cases that need more reasoning, and judging evaluation runs.

Chosen on correctness, groundedness, latency and cost per resolved conversation.

CONTRACTS AND PROOF

Typed tools and measured quality

Tool contracts

Role typed, validated tool calls

Rule schema with additionalProperties false, and validation before execution

Instead of: TypeBox, Pydantic

AI regression evaluation

Role regression suite and release gate

Rule deterministic assertions gate release, and model judges are one signal only

Instead of: DeepEval, a custom runner

LLM tracing

Role replayable traces of model calls

Rule redact before sending, and sample by policy

Instead of: LangSmith, OpenTelemetry GenAI conventions

CHAPTER 5

Clients, channels and identity

What customers and staff touch, and how we know who they are.

INTERFACES

Widget, console and live transport

Preact widget

Role customer chat on the website

Rule Shadow DOM isolation, no secrets in the bundle, works under a strict CSP

Instead of: React

React console

Role agent inbox and admin screens

Rule the UI hides, the API denies

Instead of: Vue, Angular

WebSocket and SSE

Role live messages

Rule persist first, push second

Instead of: long polling

IDENTITY AND CHANNELS

Login, sessions and later channels

OIDC provider

Role staff login

Rule no password handling in our code, authorization stays in the app

Instead of: Ory, WorkOS

JWT sessions

Role widget session for customers

Rule short lifetime, scoped claims, verified on every call

Instead of: opaque tokens

Email and messaging adapters

Role later channels

Rule same canonical event contract as website chat

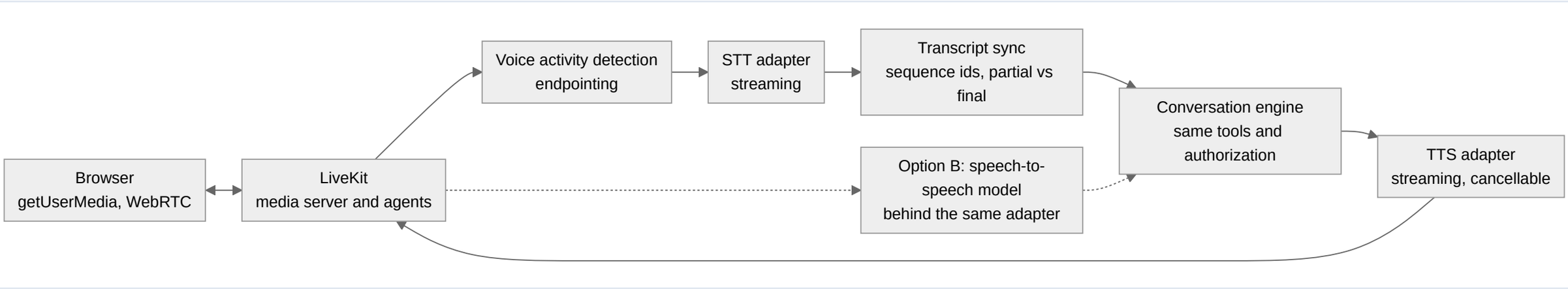
Instead of: other providers

CHAPTER 6

Voice

Real-time voice on the same engine and the same authorization.

The voice stack



Partial transcripts stay ephemeral. Only finalized speech becomes a message.

VOICE COMPONENTS

Media, speech and the alternative

LiveKit and WebRTC

Role real-time audio

Rule access tokens are short-lived and scoped to one conversation

Instead of: Twilio Voice, Daily

STT and TTS adapters

Role speech in and speech out

Rule cancel on barge-in, and keep no raw audio by default

Instead of: Whisper self-hosted

Speech-to-speech

Role low-latency voice option

Rule tool calls still pass through our authorization

Instead of: the modular pipeline

SPIKE IN SPRINT 7

How we will decide

- End-of-speech to response latency
- Language coverage
- Cost per minute
- Control over the transcript
- Privacy and retention options
- Handoff behavior for the chosen provider

CHAPTER 7

Integrations and security

Reaching business systems without handing over the keys.

REACHING OUT

Connectors, secrets and inbound events

Connector adapters

Role reach business systems

Rule one interface, typed errors, timeout and idempotency

Instead of: vendor SDKs inside adapters

Secret manager

Role credentials

Rule reference by id, never in logs, prompts or the repository

Instead of: Vault, Doppler

Webhook verification

Role trust inbound events

Rule HMAC and timestamp check, deduplicate by event id

Instead of: provider SDK verifiers

DEFENSIVE LAYERS

Validation, files and resilience

Zod and Ajv

Role validation at the edges

Rule validate requests, adapter boundaries and model output

Instead of: TypeBox

File safety

Role uploads you can trust

Rule content sniffing and a malware scan before indexing, private by default

Instead of: managed scanning

Retries and breakers

Role survive flaky providers

Rule bounded attempts with jitter, and an open breaker fails fast

Instead of: a custom implementation

CHAPTER 8

Quality and delivery

Test with real databases, scan every change, ship behind flags.

TESTING AND SCANNING

What runs on every change

Vitest and Testcontainers

Role unit and integration tests

Rule use a real Postgres for tenant and authorization tests

Instead of: Jest

Playwright and k6

Role end-to-end and load tests

Rule cover widget and console critical paths, and run load tests before each release

Instead of: Cypress

Security scanners

Role static, secret and dependency scans

Rule Semgrep, gitleaks and Trivy are required checks on every merge

Instead of: Snyk, SonarQube

RUNNING IT

Observe, build, deploy, roll back

OpenTelemetry and Grafana

Role traces, metrics and logs

Rule trace id on every request and job, no personal data in labels

Instead of: Datadog

CI/CD and Docker

Role automation and packaging

Rule trunk-based, no manual production deploys, one image runs as API and worker

Instead of: GitLab CI

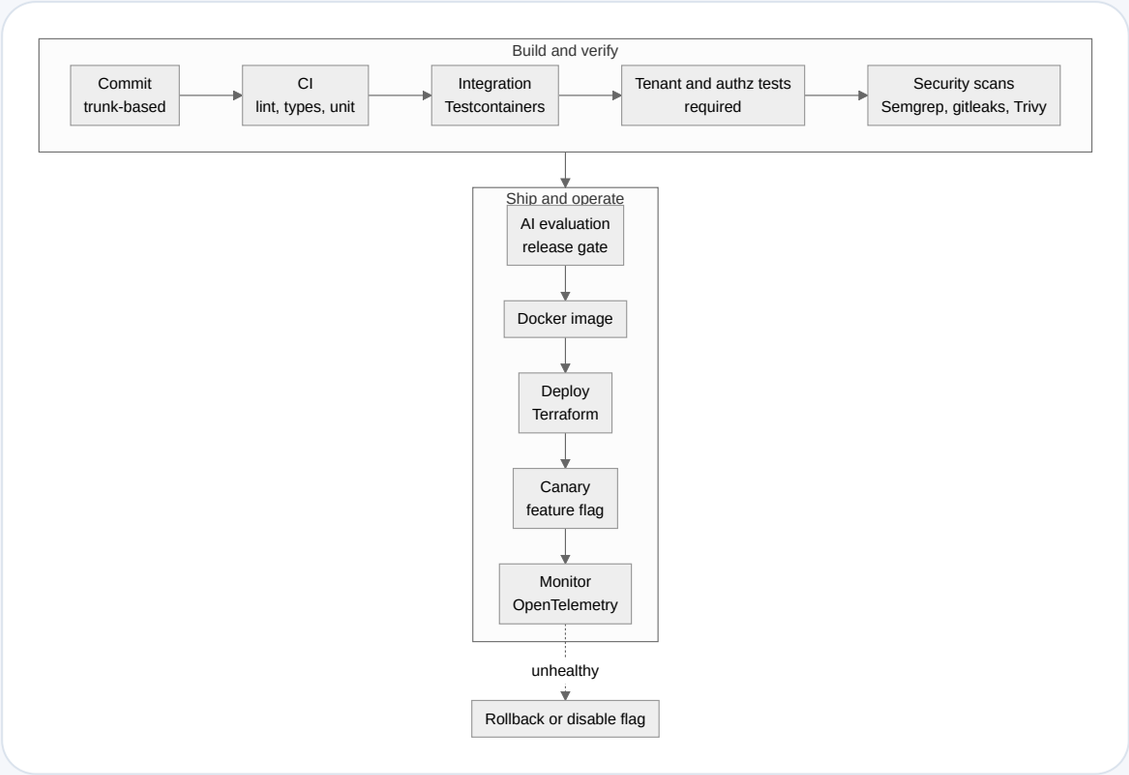
Terraform and OpenFeature

Role infrastructure and flags

Rule environments are reproducible, and every risky feature ships behind a flag

Instead of: Pulumi

From commit to production



Gates run in order. An unhealthy canary rolls back or disables the flag.

CHAPTER 9

Technology by task

Which technologies each epic and Sprint relies on.

EPICS 1 TO 6

Technology by epic

EPIC	KEY TECHNOLOGIES
E01 Tenant and access	Postgres RLS, Testcontainers, OIDC, jose tokens
E02 Conversation core	NestJS, WebSocket, Postgres sequences, outbox
E03 Website chat	Preact Web Component, JWT sessions, Redis rate limits
E04 Knowledge and RAG	S3, ClamAV, pg-boss, pgvector with full-text
E05 AI orchestration	State machine, LLM adapter, AbortController, Langfuse
E06 Safe business tools	Ajv, connector adapters, idempotency keys, audit table

EPICS 7 TO 12

Technology by epic

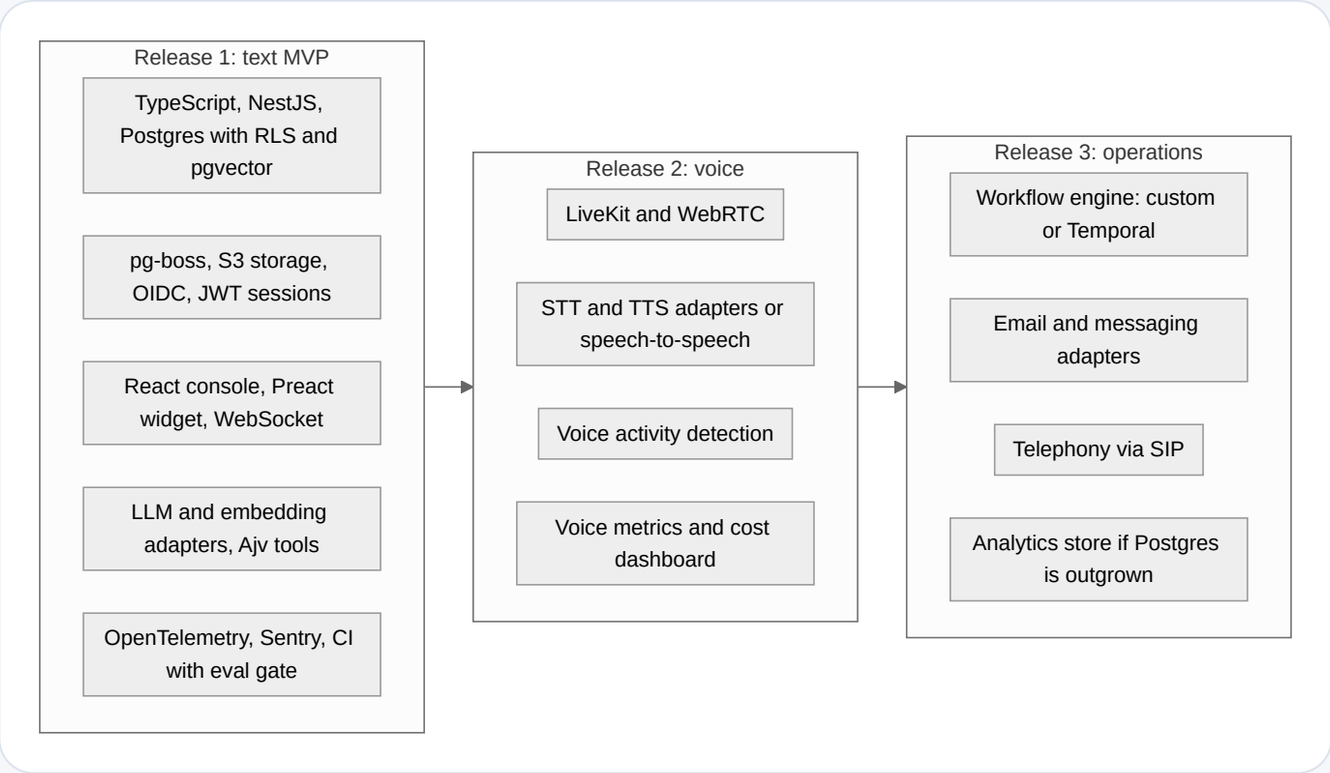
EPIC	KEY TECHNOLOGIES
E07 Human handoff	SKIP LOCKED claims, WebSocket updates, React console
E08 Tickets	Postgres, keyset pagination, TanStack Table
E09 Voice MVP	LiveKit, STT and TTS adapters, voice activity detection
E10 Workflow, routing, SLA	Workflow state machine, pg-boss timers, Temporal evaluated
E11 Integrations and channels	Secret manager, HMAC webhooks, email and messaging adapters
E12 Observability, evaluation, usage	OpenTelemetry, Grafana, promptfoo, Langfuse

ARRIVAL ORDER

Technology by Sprint

SPRINT	INTRODUCED
Sprint 0	Docker Compose, CI, Terraform skeleton, migrations, secrets
Sprint 1	NestJS, row-level security, audit table, OpenTelemetry, Testcontainers
Sprint 2	Preact widget, WebSocket gateway, JWT sessions, Playwright
Sprint 3	Object storage, ClamAV, parsers, pg-boss ingestion, pgvector
Sprint 4	LLM adapter, structured outputs, citations, evaluation harness
Sprint 5	Ajv tool registry, connector with breaker, React console inbox
Sprint 6	Dashboards, k6, backup drill, feature flags, evaluation gate
Sprint 7	LiveKit, voice activity detection, STT and TTS or speech-to-speech
Sprint 8	Transcript sync, barge-in cancellation, cost and latency dashboard

When each technology arrives



Release 1 carries the core stack, voice adds real-time pieces, operations adds channels and workflows

PER STORY

43 stories, each with its technologies

The report lists the technologies and the approach for every user story and every Sprint.

See the rapport, Part V, chapter "Technology by Task".

CHAPTER 10

Decisions and risks

What we still have to validate, and how we limit lock-in.

VALIDATE WITH EVIDENCE

Decisions to make first

DECISION	OPTIONS TO COMPARE	WHEN
LLM provider and routing	Two or three providers and model sizes	Sprint 4
Vector search	pgvector against a dedicated engine	Sprint 3
Queue and outbox	pg-boss against Redis with BullMQ	Sprints 0 and 1
Authentication	Keycloak, Auth0, Cognito	Sprint 1
Hosting	Managed containers, Fly.io or Render, Kubernetes	Sprint 0

VALIDATE WITH EVIDENCE

Decisions to make later

DECISION	OPTIONS TO COMPARE	WHEN
Speech strategy	Modular STT and TTS against speech-to-speech	Sprint 7
Voice transport and telephony	LiveKit, Twilio, Daily	Sprint 7
Retention and consent	Audio and transcript retention models	Sprint 7
Workflow engine	Custom state machine against Temporal	Release 3

KEEPING OPTIONS OPEN

Lock-in and mitigation

TECHNOLOGY	LOCK-IN	MITIGATION
LLM provider	Medium	Adapter, provider-neutral tool schemas, evaluation suite to compare models
Embedding model	Medium	Store the model version per chunk and keep a re-index job
Speech and voice	High for speech-to-speech	One strategy first, thin adapters, measure before committing
PostgreSQL and pgvector	Low to medium	Standard SQL, retrieval behind an interface, documented switch trigger
Cloud services	Medium	Containers, Terraform modules, S3-compatible API
Observability and flags	Low	OpenTelemetry and OpenFeature as the client layers

Choose boring where it matters

Spend the novelty on AI behavior, and keep everything else proven, portable and observable.

Full detail: rapport.pdf, Part V.